

In relational databases, a JOIN operation is used to combine rows from two or more tables based on a related column between them. Understanding the different types of JOINS is essential for retrieving meaningful data from multiple tables efficiently.

Inner Joins:

Inner joins return only the rows that have matching values in both tables.

- **Theta Join:**
 1. This is the most general form of join.
 2. It allows you to use any comparison operator ($=$, $<$, $>$, $<=$, $>=$, $\neq$) in the join condition.
 3. **Example:** `SELECT * FROM A, B WHERE A.id > B.id;`
- **EQUI Join:**
 1. A special case of theta join where only the equality operator ($=$) is used.
 2. It returns rows with matching values in the specified columns of both tables.
 3. **Example:** `SELECT * FROM A, B WHERE A.id = B.id;`
- **Natural Join:**
 1. Automatically matches columns between tables with the same name and data type.
 2. Eliminates duplicate columns in the result.
 3. **Example:** `SELECT * FROM A NATURAL JOIN B;`
- **Inner Join** = Only matched data
- **Left/Right Outer Join** = All data from one side + matched data
- **Full Outer Join** = All data from both sides + NULLs where unmatched

Table 1: Employees

EmpID	EmpName	DeptID
101	Alice	D1
102	Bob	D2
103	Charlie	D3
104	Diana	NULL
105	Ethan	D2
106	Farah	D5

 **Table 2: Departments**

DeptID	DeptName
D1	HR
D2	Engineering
D3	Marketing
D4	Finance

INNER JOIN:

An `INNER JOIN` is used to combine rows from two or more tables **based on a matching condition**. It returns only those rows where there is a **match in both tables**.

Syntax:

`SELECT columns`

`FROM TableA`

`INNER JOIN TableB`

`ON TableA.common_column = TableB.common_column;`

How It Works:

- The `INNER JOIN` compares each row of `TableA` with each row of `TableB`.
- It checks if the join condition is satisfied (usually an equality between keys).
- Only those row pairs where the condition is **true** are included in the final result.
- If there is **no match**, the row is **excluded** from the result

1. Theta Join

A **Theta Join** is a type of join that allows **arbitrary comparison operators** in the join condition, such as `=`, `>`, `<`, `!=`, `>=`, `<=`.

SELECT *

FROM TableA, TableB

WHERE TableA.column θ TableB.column;

(Where θ can be any comparison operator)

Example:

SELECT EmpName, DeptName

FROM Employees, Departments

WHERE Employees.DeptID != Departments.DeptID;

EmpID	EmpName	DeptID
101	Alice	D1
102	Bob	D2
103	Charlie	D3

EmpID	EmpName	DeptID
DeptID	DeptName	
D1	HR	
D2	Engineering	
D3	Marketing	

SELECT EmpName, DeptName FROM Employees, Departments

WHERE Employees.DeptID != Departments.DeptID;

EmpName	DeptName
Alice	Engineering
Alice	Marketing
Bob	HR
Bob	Marketing
Charlie	HR
Charlie	Engineering

2. Equi Join

An **Equi Join** is a **Theta Join** where only the **equality operator (=)** is used in the join condition. It's the most common type of join used in relational databases.

```
SELECT *  
  
FROM TableA, TableB  
  
WHERE TableA.column = TableB.column;
```

```
SELECT EmpName, DeptName  
  
FROM Employees, Departments  
  
WHERE Employees.DeptID = Departments.DeptID;
```

EmpName	DeptName
Alice	HR
Bob	Engineering
Charlie	Marketing

3.Natural Join

A **Natural Join** automatically joins tables based on **all columns with the same name and data type**, and **eliminates duplicates** in the result. You don't need to specify the condition.

```
SELECT *  
  
FROM TableA  
  
NATURAL JOIN TableB;
```

```
SELECT EmpName, DeptName  
  
FROM Employees  
  
NATURAL JOIN Departments;
```

EmpName	DeptName
Alice	HR
Bob	Engineering
Charlie	Marketing

❗ **Note:** Duplicate DeptID column is removed in the result.

JOIN Type	Match Condition	Duplicate Columns?	Auto Match?	Use Case
Theta Join	Any comparison (=, <, !=, etc.)	Yes	No	Custom filter (e.g., salary > 50000)
Equi Join	Equality (=) only	Yes	No	Most common form, matches foreign key
Natural Join	Automatically on all columns with the same name	No (removes duplicates)	Yes	Simplified join when schema is designed consistently

- Use **INNER JOIN** for most foreign key-based queries.
- Use **OUTER JOINS** when you need **all data from one or both tables**, even if there are **no matches**.
- Use **NATURAL JOIN** only when you're sure the column names and types match across tables.

Outer Joins:

Outer joins return all the matching rows plus any unmatched rows from one or both tables, filling in NULLs where necessary.

- **Left Outer Join:**
 1. Returns all rows from the left table, and the matched rows from the right table.
 2. If no match is found, NULLs are returned for columns from the right table.
 3. **Example:** `SELECT * FROM A LEFT JOIN B ON A.id = B.id;`
- **Right Outer Join:**
 1. Returns all rows from the right table, and the matched rows from the left table.
 2. If no match is found, NULLs are returned for columns from the left table.
 3. **Example:** `SELECT * FROM A RIGHT JOIN B ON A.id = B.id;`
- **Full Outer Join:**
 1. Returns all rows when there is a match in one of the tables.
 2. Rows with no match in either table will have NULLs for missing values.
 3. **Example:** `SELECT * FROM A FULL OUTER JOIN B ON A.id = B.id;`

EmpID	EmpName
1	Alice
2	Bob

EmpID	EmpName
3	Charlie
4	Diana

EmpID	Salary
2	50000
3	60000
5	70000

1. LEFT OUTER JOIN

SELECT Employees.EmpName, Salaries.Salary

FROM Employees

LEFT JOIN Salaries ON Employees.EmpID = Salaries.EmpID;

EmpName	Salary
Alice	NULL
Bob	50000
Charlie	60000
Diana	NULL

2. RIGHT OUTER JOIN:

SELECT Employees.EmpName, Salaries.Salary

FROM Employees

RIGHT JOIN Salaries ON Employees.EmpID = Salaries.EmpID;

EmpName	Salary
Bob	50000
Charlie	60000
NULL	70000

4. FULL OUTER JOIN:

SELECT Employees.EmpName, Salaries.Salary

FROM Employees

FULL OUTER JOIN Salaries ON Employees.EmpID = Salaries.EmpID;

EmpName	Salary
Alice	NULL
Bob	50000
Charlie	60000
Diana	NULL
NULL	70000

Note: Some databases (like MySQL) don't support `FULL OUTER JOIN` directly. You can simulate it using `UNION`

`SELECT * FROM A LEFT JOIN B ON A.id = B.id`

`UNION`

`SELECT * FROM A RIGHT JOIN B ON A.id = B.id;`